
CLOUDMART

Highly Available E-Commerce Application on AWS

Project Documentation

Cloud Platform: Amazon Web Services (AWS)
Region: Asia Pacific (Hyderabad) — ap-south-2
Prepared by: Yabesh
Date: 29 August 2026

Contents

1. Executive Summary
2. Architecture Overview
3. AWS Services Used
4. Infrastructure Verification
5. Application Walkthrough — Customer Experience
6. Application Walkthrough — Admin Experience
7. Operational Verification
8. Security Practices Applied
9. Conclusion

1. Executive Summary

CloudMart is a highly available, three-tier e-commerce web application built entirely on AWS. It demonstrates a production-style architecture — a public web tier behind a load balancer, an auto-scaling application tier, and a Multi-AZ managed database — combined with serverless building blocks (S3, SQS, Cognito) for storage, asynchronous processing, and authentication.

The application allows guests to browse a product catalog without an account, requires authentication for checkout, and gives administrators a dedicated dashboard to approve or reject incoming orders. Order status updates automatically flow through to the customer's "My Orders" page.

Key Capabilities

Capability	Status
Public storefront (browse without login)	Implemented
Role-based navigation (guest / user / admin)	Implemented
User sign-up, email confirmation, login (Cognito)	Implemented
Cart & checkout (login required)	Implemented
Order placed → SQS → asynchronous processing	Implemented
Admin order approval / rejection workflow	Implemented
Order status visible to customer in real time	Implemented
Multi-AZ high availability (ALB + ASG + Aurora)	Implemented

2. Architecture Overview

The application follows a classic three-tier design deployed inside a dedicated VPC spanning two Availability Zones for fault tolerance. Public traffic reaches an Application Load Balancer, which distributes requests across two EC2 instances managed by an Auto Scaling Group. The application tier connects to a Multi-AZ Aurora MySQL cluster for persistent data, Amazon S3 for product images and deployment artifacts, and Amazon SQS to decouple order placement from order processing. Amazon Cognito handles user identity and role-based access (Users vs. Admins).

The diagram below maps every AWS service used in this project to its position in the network — from the public-facing load balancer down to the private database tier.

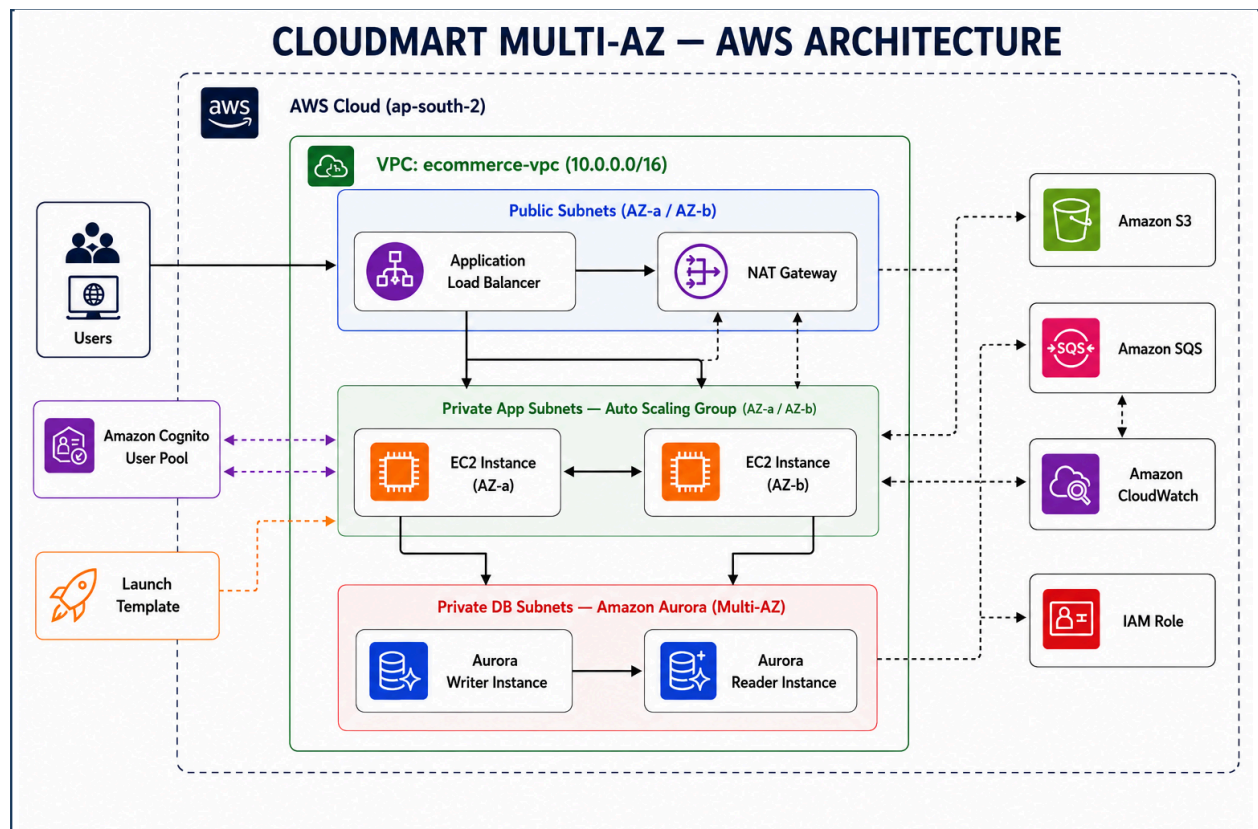


Figure 1 — CloudMart AWS architecture diagram

Request Flow

- A user's browser sends a request to the ALB's public DNS name.
- The ALB forwards the request to a healthy EC2 instance in the target group.
- Flask serves the storefront publicly; checkout, orders and admin routes require Cognito login.
- On checkout, Aurora gets a PENDING order row and a message is published to SQS.
- A background worker consumes the SQS message and moves the order to PROCESSING.
- An admin approves or rejects PROCESSING orders; the update appears on the customer's My Orders page instantly.

3. AWS Services Used

AWS Service	Role in Architecture
Amazon VPC	Isolated network — 6 subnets across 2 AZs (public, private-app, private-db)
Application Load Balancer	Public entry point; distributes HTTP traffic across the Auto Scaling Group
EC2 Auto Scaling Group	Runs the Flask application (Gunicorn) and the SQS worker; self-heals via Instance Refresh
Amazon Aurora (MySQL, Multi-AZ)	Stores products, orders and order_items with a writer + reader instance
Amazon S3	Hosts product images (served via presigned URLs) and the deployment source ("deploy/app/")
Amazon SQS	Decouples checkout from order processing; worker consumes messages asynchronously
Amazon Cognito	User authentication (sign-up / login) and role management via an "Admins" group
IAM	Least-privilege instance role for EC2 → S3 / SQS access

4. Infrastructure Verification

The following screenshots confirm each AWS resource is provisioned and healthy.

4.1 VPC & Subnets

The network foundation: a single VPC with six subnets split evenly across two Availability Zones, giving every tier (web, app, database) redundancy if one zone fails.

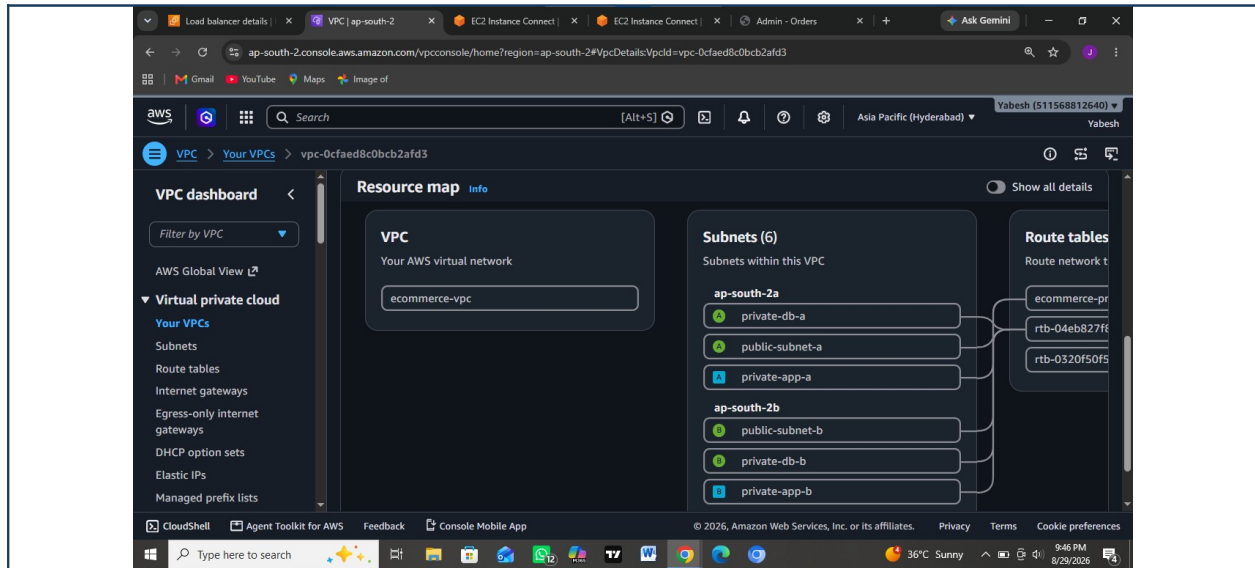


Figure 2 — VPC with 6 subnets across two Availability Zones

4.2 Auto Scaling Group

The Auto Scaling Group keeps exactly two application instances running at all times, automatically replacing any instance that fails a health check.

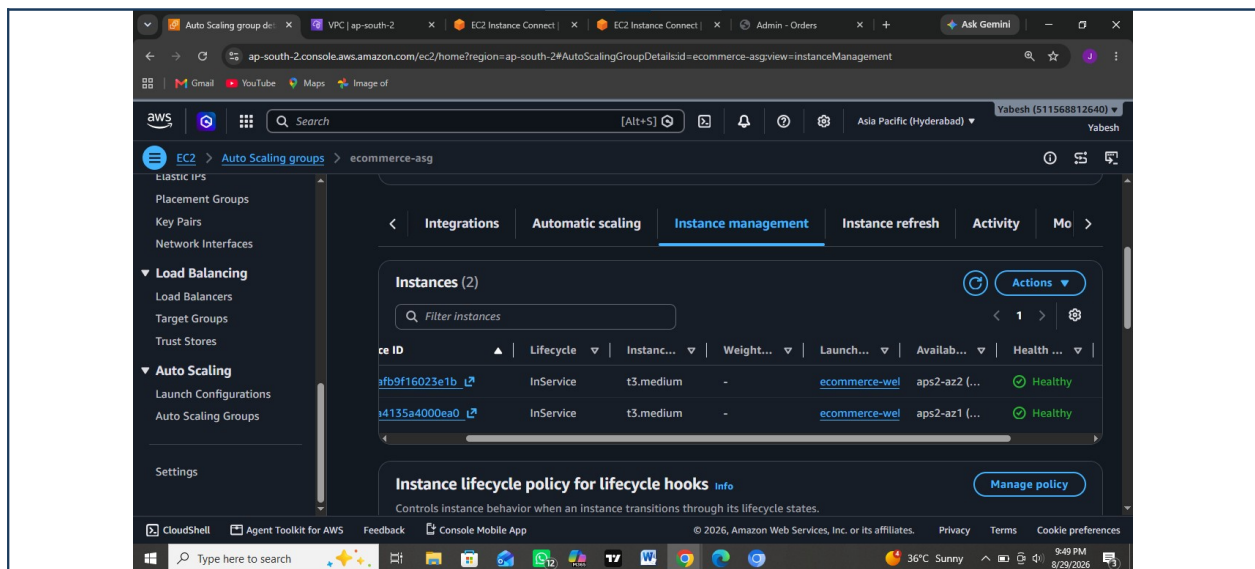


Figure 3 — Auto Scaling Group with two InService, healthy EC2 instances

4.3 Target Group Health

The load balancer's target group confirms both EC2 instances are passing HTTP health checks and are eligible to receive live traffic.

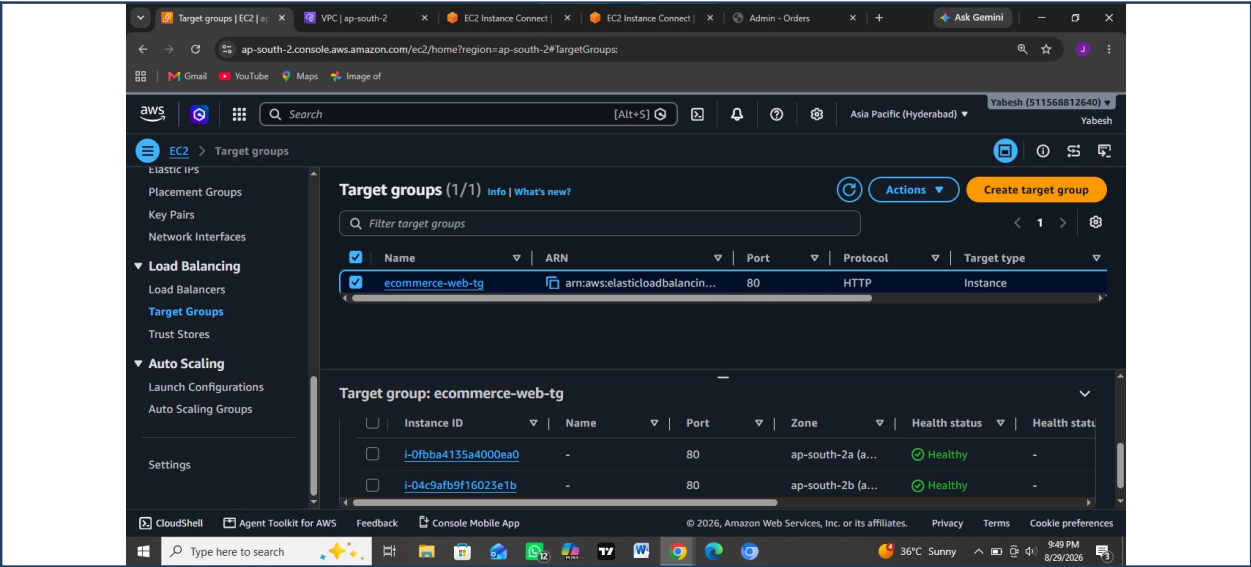


Figure 4 — Target group reporting both instances healthy

4.4 Application Load Balancer

The Application Load Balancer is the single public entry point for the application, spanning two Availability Zones for resilience.

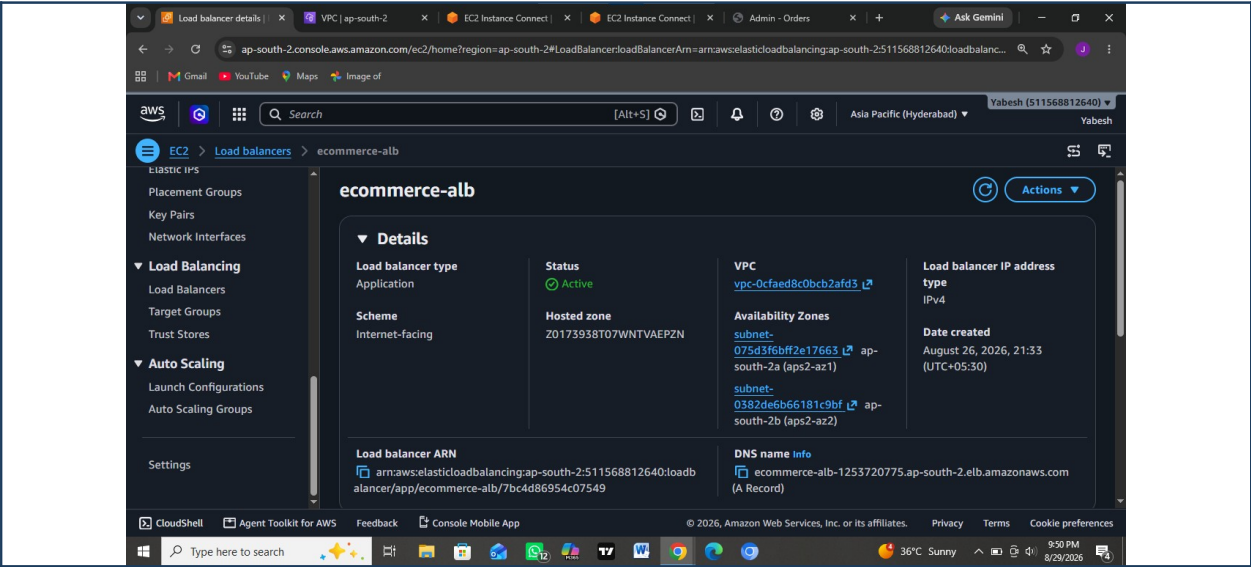


Figure 5 — ALB details, Internet-facing, spanning two Availability Zones

4.5 Amazon Aurora (Multi-AZ)

The data tier runs on Amazon Aurora MySQL with a dedicated writer instance and a Multi-AZ reader for automatic failover.

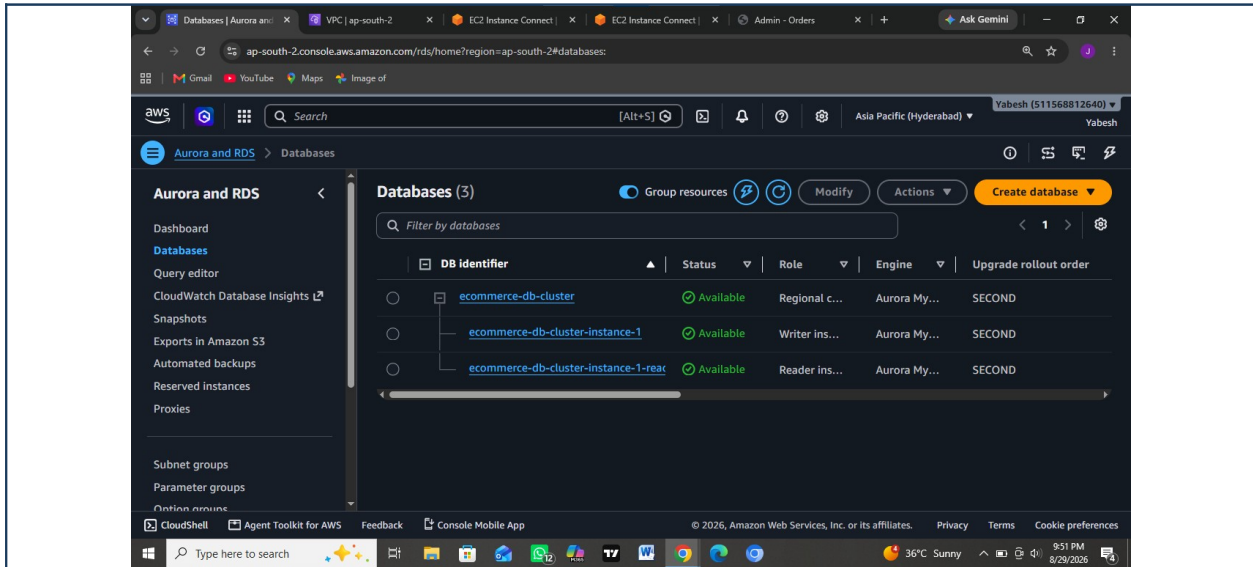


Figure 6 — Aurora cluster with writer and reader instances

4.6 Amazon S3

S3 serves two purposes here: storing the deployable application source ("deploy/app/") that new instances boot from, and hosting product images.

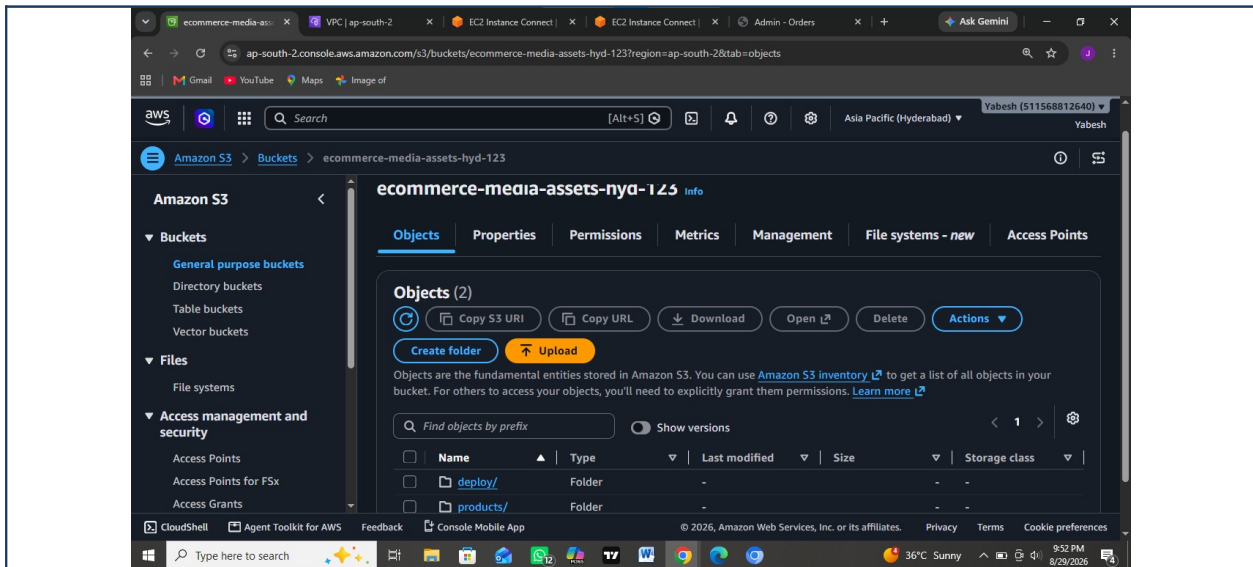


Figure 7 — S3 bucket holding deployment source and product images

4.7 Amazon SQS

A standard SQS queue decouples order placement from order processing, so checkout stays fast while the order is handled asynchronously.

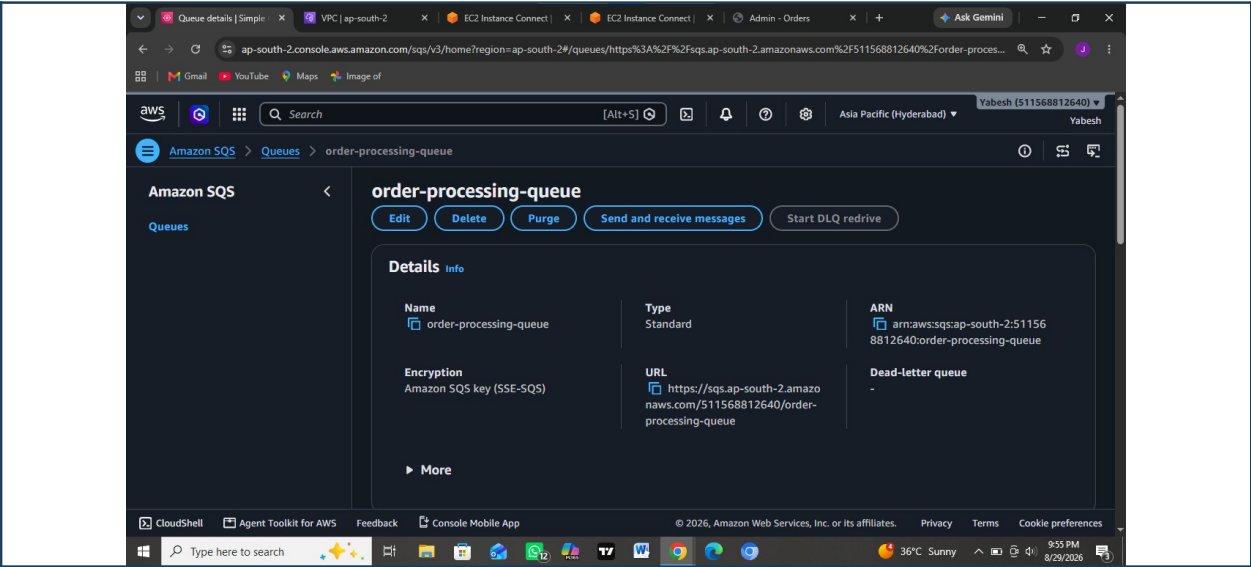


Figure 8 — order-processing-queue configuration

CloudWatch metrics confirm the queue is actively receiving and delivering messages as orders are placed.

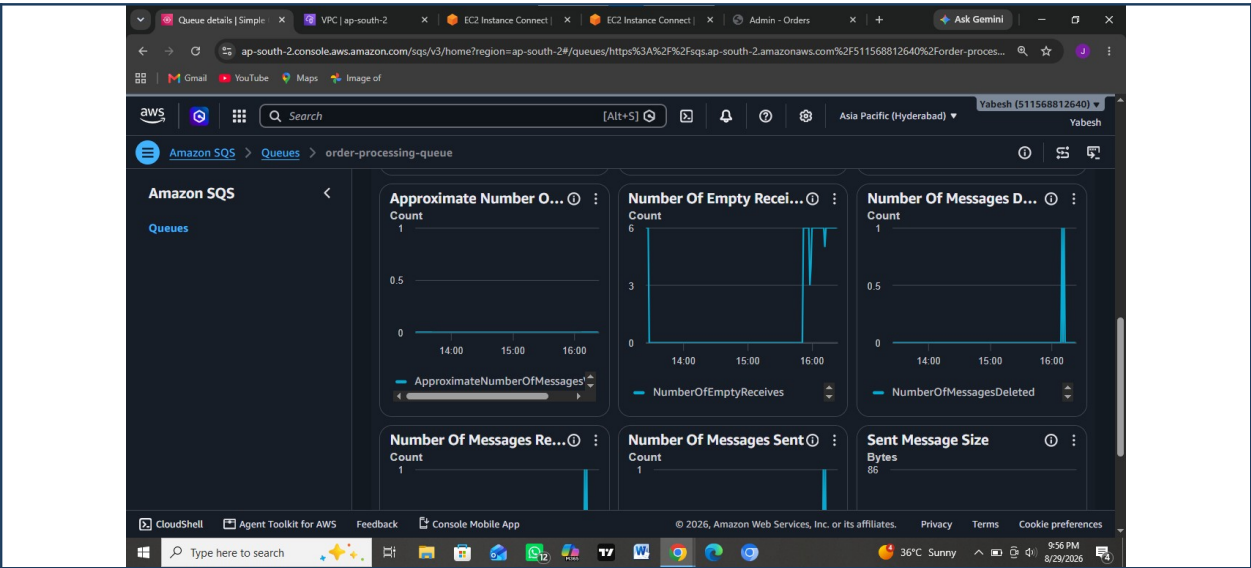


Figure 9 — SQS CloudWatch metrics showing message activity

4.8 Amazon Cognito

Amazon Cognito manages all user identity — sign-up, email confirmation, and login — removing the need for custom authentication code.

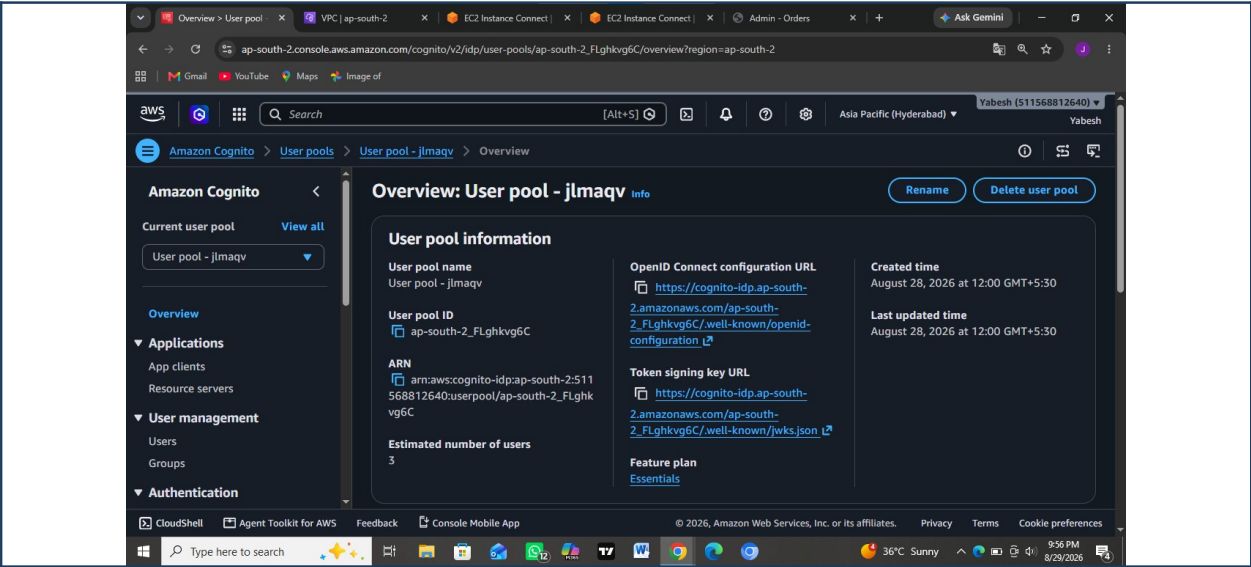


Figure 10 — Cognito User Pool overview

A dedicated "Admins" group inside the same user pool is what distinguishes an administrator from a regular customer at login time.

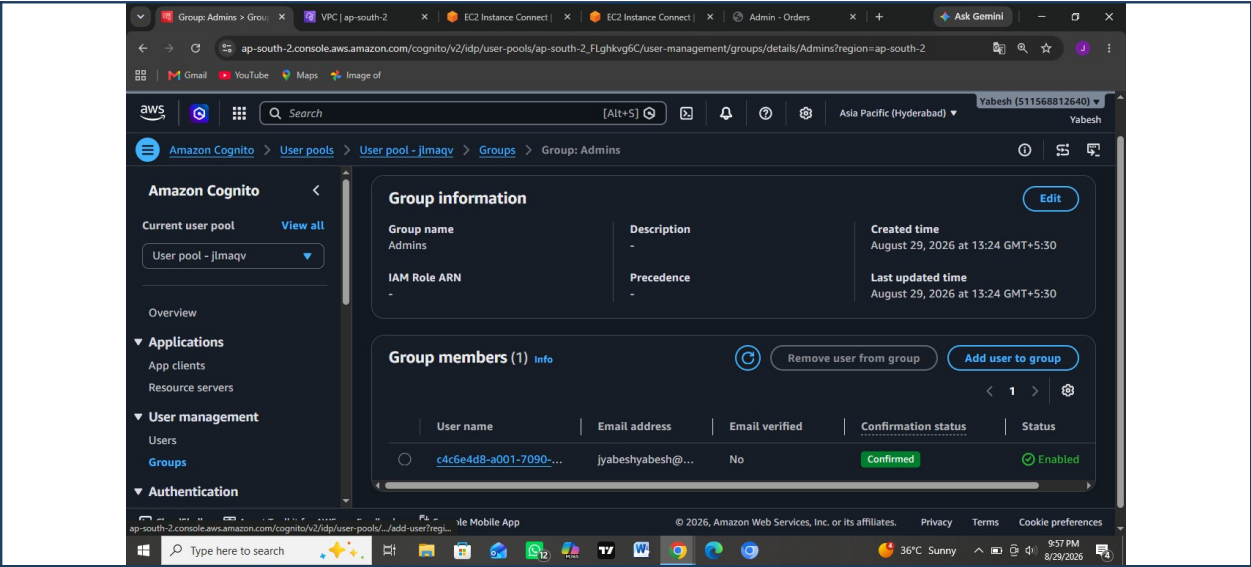


Figure 11 — "Admins" group used for role-based access control

5. Application Walkthrough — Customer Experience

A guest can browse the storefront freely. Authentication is required only at checkout.

5.1 Public Storefront

Anyone can land on the storefront and browse the catalog without creating an account or signing in.

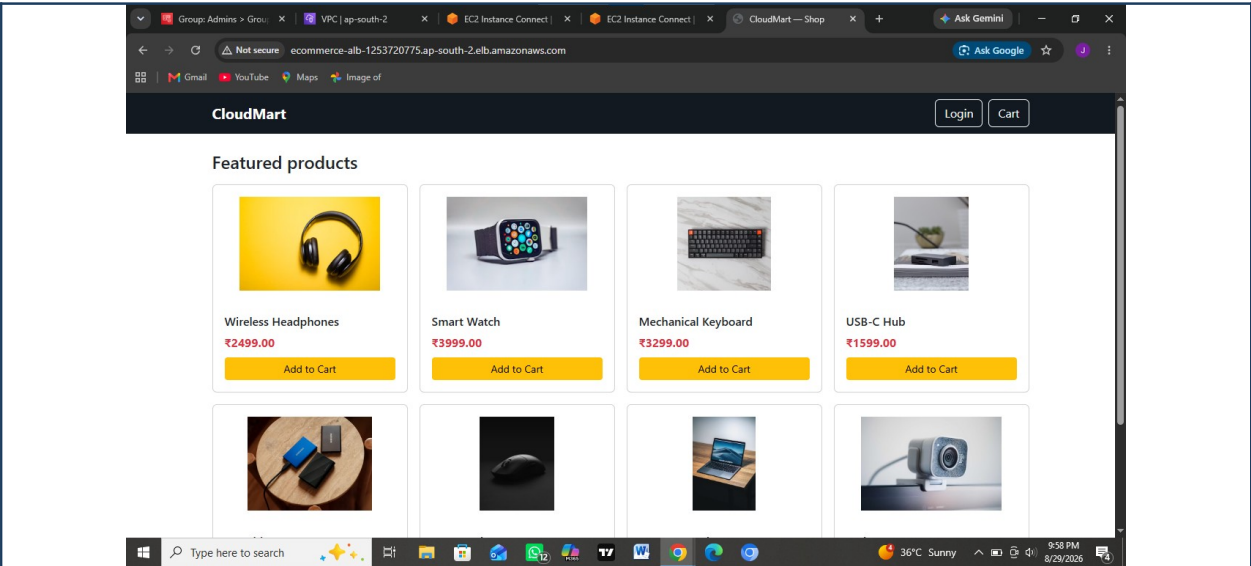


Figure 12 — Home page, accessible without logging in

5.2 Shopping Cart

Items can be added to the cart and quantities adjusted while still browsing as a guest.

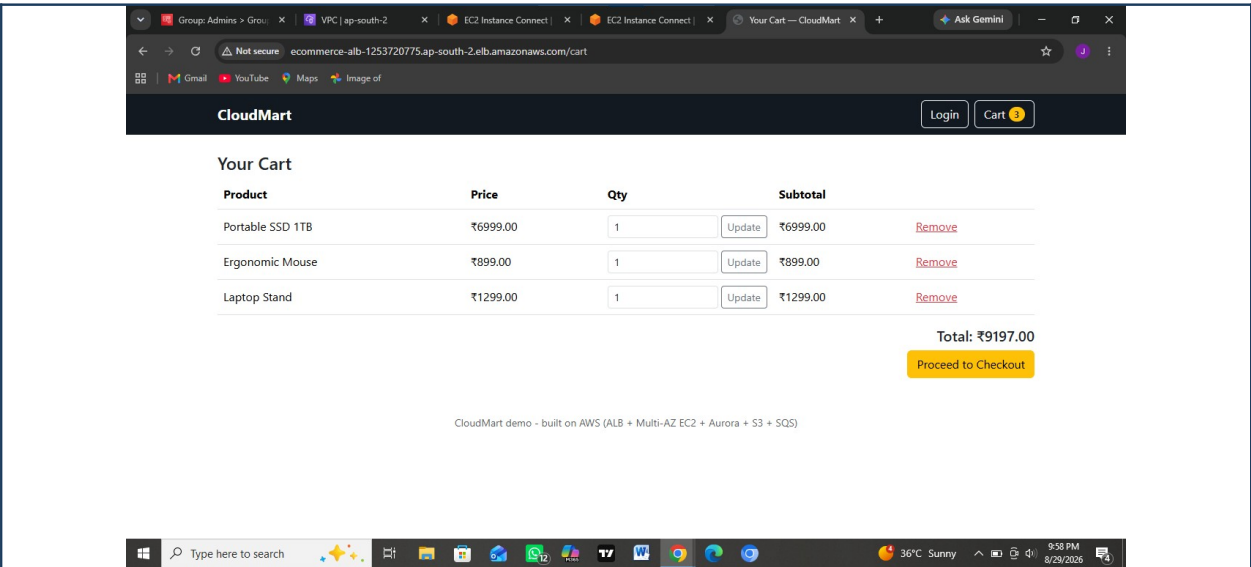


Figure 13 — Cart with items and running total

5.3 Login

The moment a guest tries to check out, they are redirected to log in; browsing itself never required an account.

CloudMart

Login

Username or Email

mornmoen021@gmail.com

Password

.....

Login

No account? [Sign up](#)

CloudMart demo - built on AWS (ALB + Multi-AZ EC2 + Aurora + S3 + SQS)

Figure 14 — Login is required only when proceeding to checkout

5.4 Checkout

Once authenticated, the customer completes shipping details against the same cart contents carried over from checkout.

CloudMart

My Orders Cart 3 Logout

Checkout

Full name

Morn

Email

mornmoen021@gmail.com

Shipping address

No 7/Calfornia states yyy northern carolina 70052

Place Order

Order summary

Portable SSD 1TB × 1	₹6999.00
Ergonomic Mouse × 1	₹899.00
Laptop Stand × 1	₹1299.00
Total: ₹9197.00	

CloudMart demo - built on AWS (ALB + Multi-AZ EC2 + Aurora + S3 + SQS)

Figure 15 — Checkout form with order summary

5.5 Order Confirmation

The order is written to the database and a message is published to SQS immediately, so the customer gets an instant confirmation.

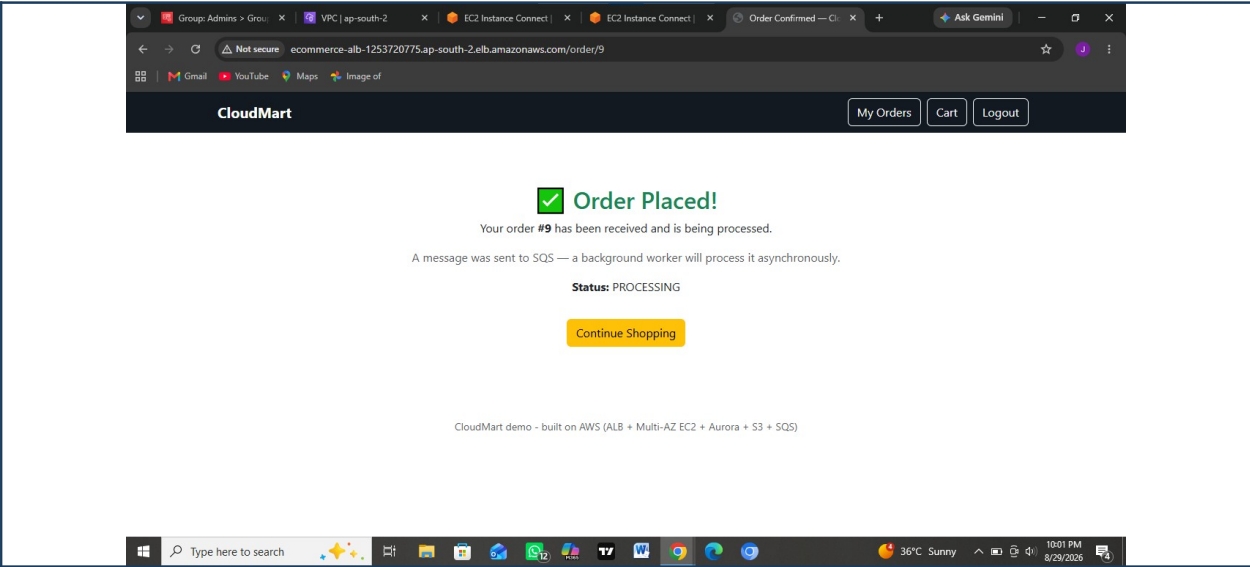


Figure 16 — Order confirmed; message published to SQS for async processing

5.6 Order History

The customer's "My Orders" page lists every past order with its live status, sourced directly from the database.

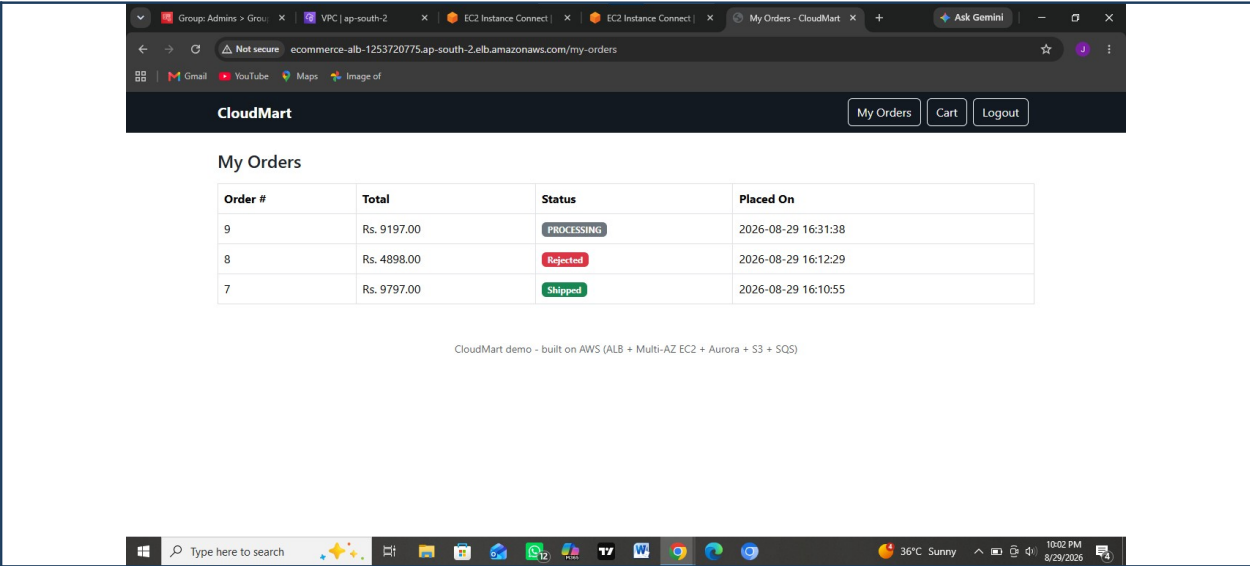


Figure 17 — "My Orders" showing live status per order

6. Application Walkthrough — Admin Experience

Administrators are members of the Cognito "Admins" group. Once signed in, the navigation bar simplifies to a single Logout action, and the admin lands on the order management dashboard.

6.1 Admin Login

The same login form is used for everyone; it is Cognito group membership, not a separate URL or password, that grants admin access.

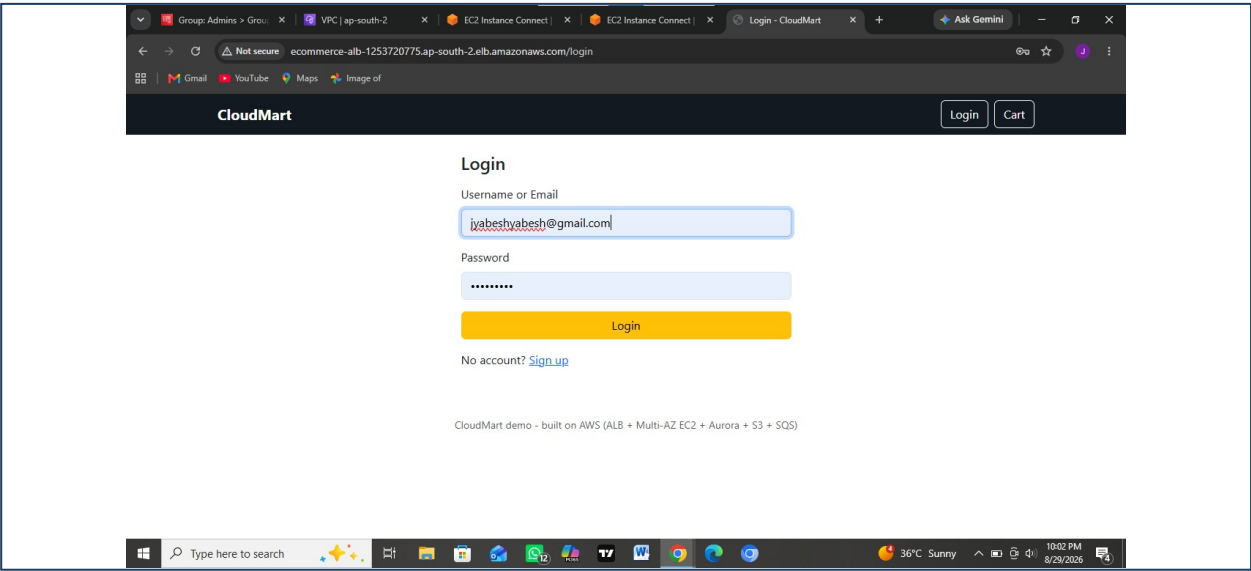


Figure 18 — Admin signs in with the designated admin account

6.2 Admin Order Dashboard

Every order the background worker has picked up appears here as PROCESSING, with Approve and Reject actions available to the admin.

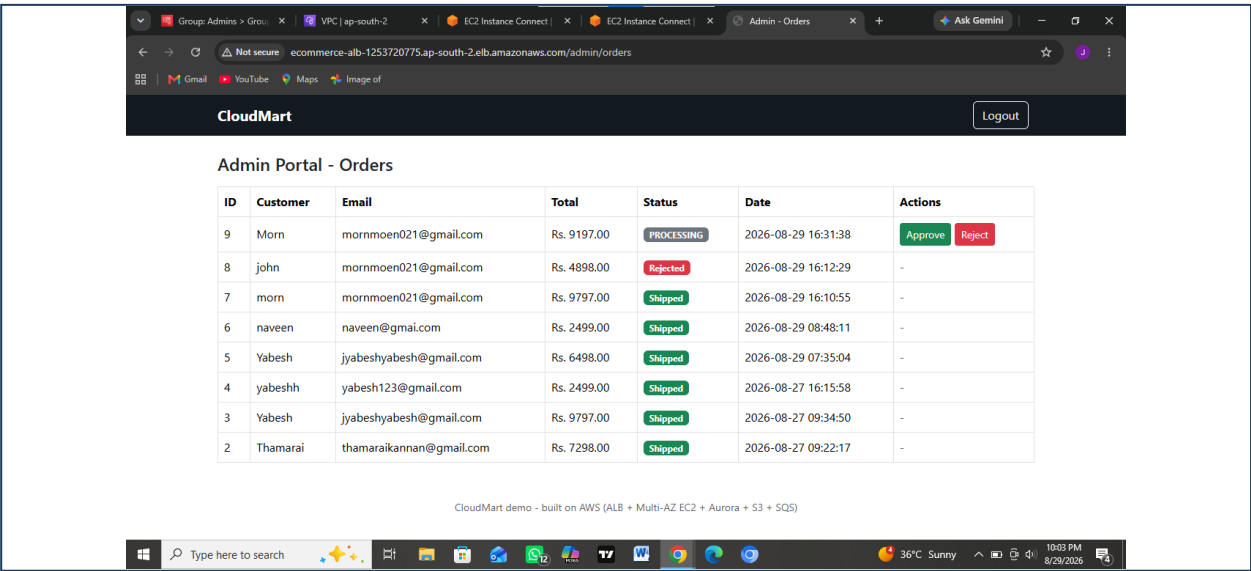


Figure 19 — Orders awaiting approval, with Approve / Reject actions

6.3 Approving an Order

Clicking Approve is the action that finally marks an order Shipped — the worker itself deliberately stops short of doing this automatically.

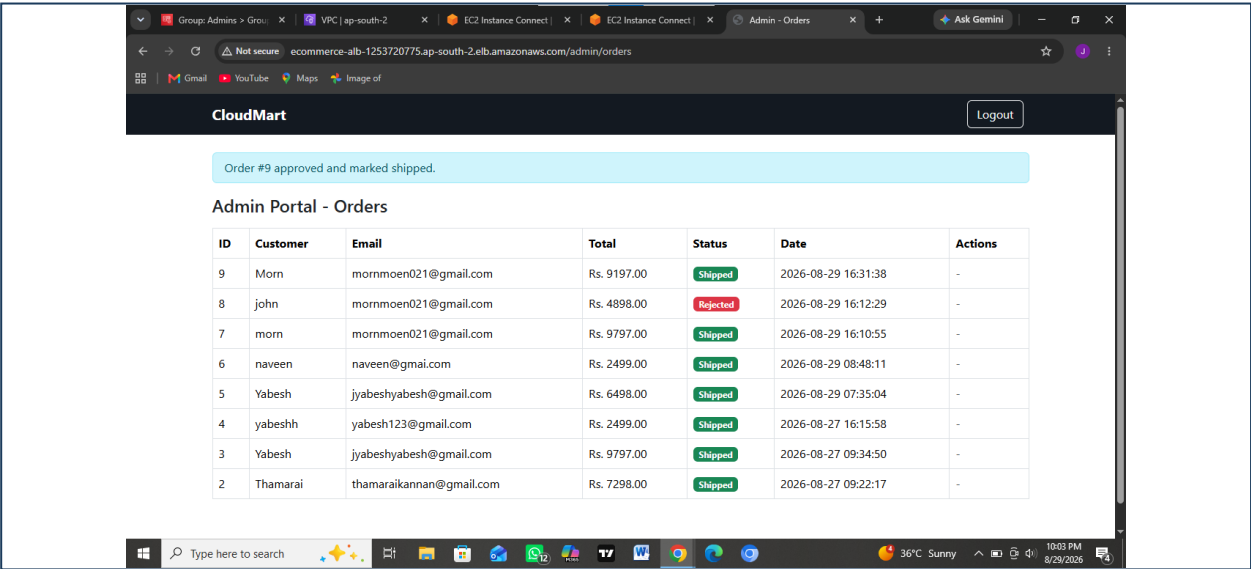


Figure 20 — Order approved and marked Shipped

6.4 Customer View Updates Automatically

Because both pages read from the same orders table, the customer sees the Shipped status the moment the admin approves it — no refresh logic needed.

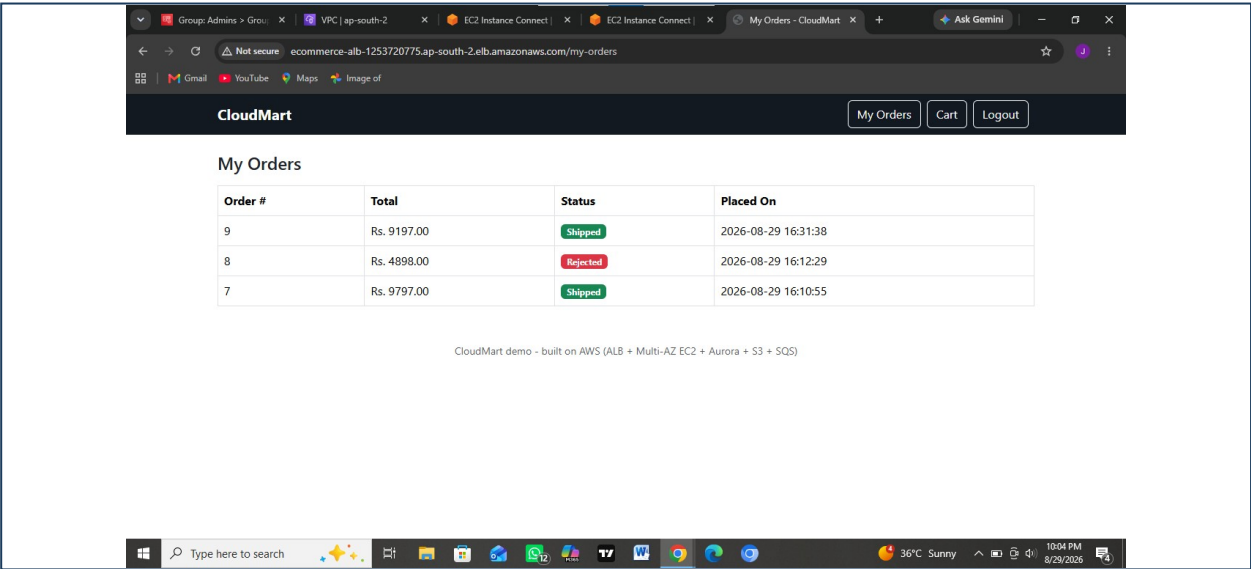


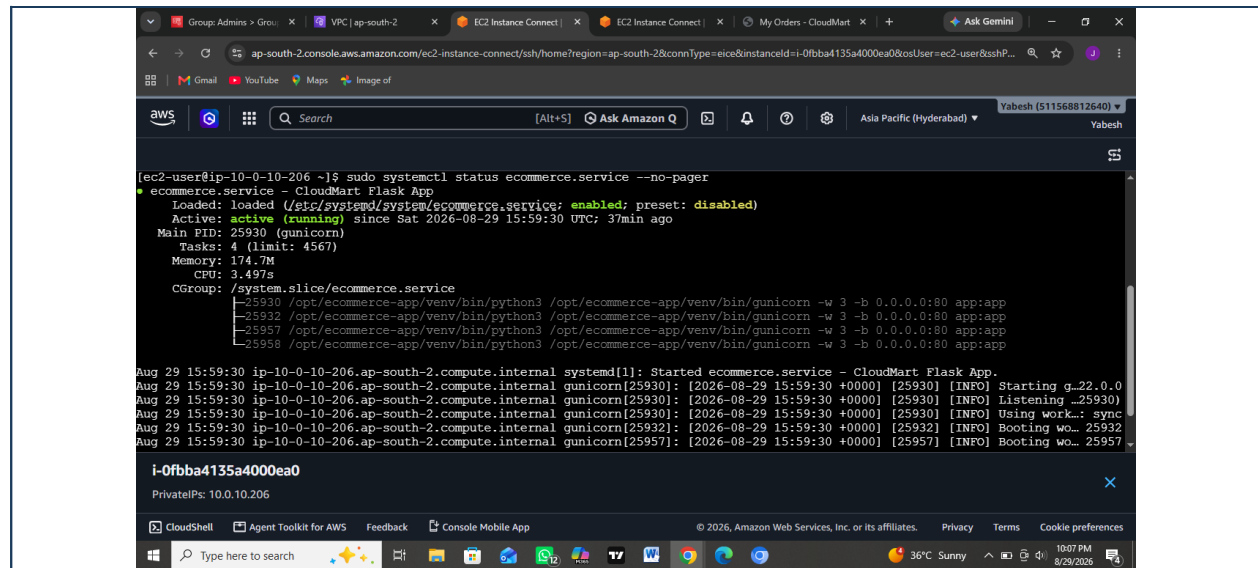
Figure 21 — The customer's My Orders page reflects the admin's decision immediately

7. Operational Verification

Both application services were confirmed active on the running EC2 instances, with a clean health check response and logs.

7.1 Web Application Service

The Flask application runs under Gunicorn as a systemd service, so it restarts automatically on failure or instance reboot.



```

[ec2-user@ip-10-0-10-206 ~]$ sudo systemctl status ecommerce.service --no-pager
● ecommerce.service - CloudMart Flask App
   Loaded: loaded (/etc/systemd/system/eccommerce.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-29 15:59:30 UTC; 37min ago
     Main PID: 25930 (gunicorn)
       Tasks: 4 (limit: 4567)
      Memory: 174.7M
         CPU: 3.497s
    CGroup: /system.slice/eccommerce.service
            └─25930 /opt/eccommerce-app/venv/bin/python3 /opt/eccommerce-app/venv/bin/gunicorn -w 3 -b 0.0.0.0:80 app:app
              └─25932 /opt/eccommerce-app/venv/bin/python3 /opt/eccommerce-app/venv/bin/gunicorn -w 3 -b 0.0.0.0:80 app:app
              └─25957 /opt/eccommerce-app/venv/bin/python3 /opt/eccommerce-app/venv/bin/gunicorn -w 3 -b 0.0.0.0:80 app:app
              └─25958 /opt/eccommerce-app/venv/bin/python3 /opt/eccommerce-app/venv/bin/gunicorn -w 3 -b 0.0.0.0:80 app:app

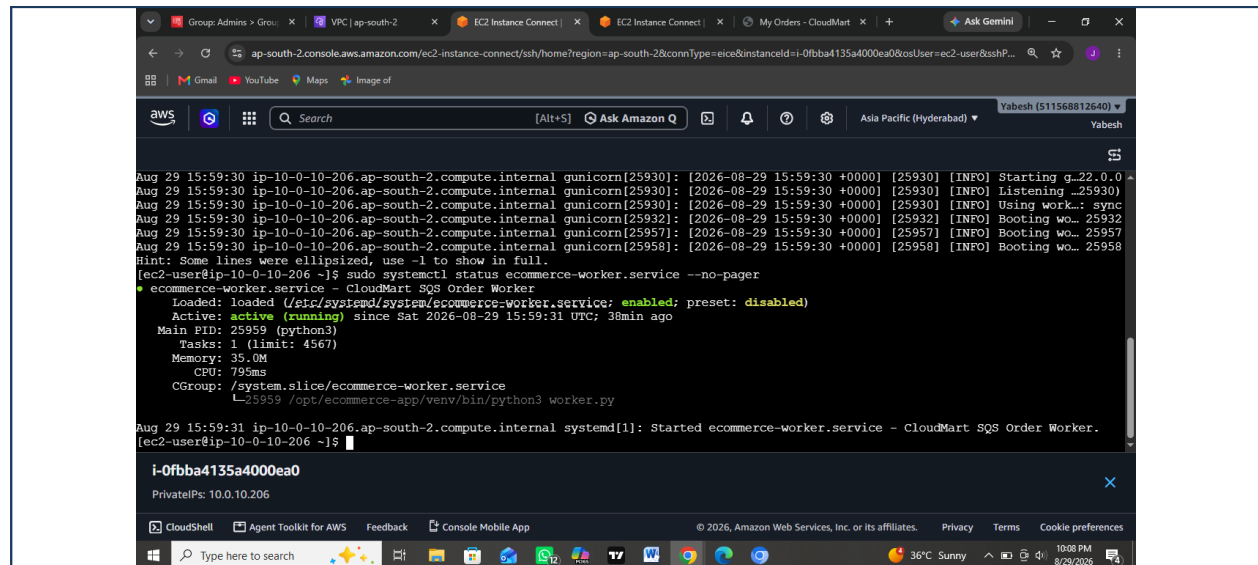
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal systemd[1]: Started ecommerce.service - CloudMart Flask App.
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25930]: [2026-08-29 15:59:30 +0000] [25930] [INFO] Starting g...2.0.0
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25930]: [2026-08-29 15:59:30 +0000] [25930] [INFO] Listening ...25930)
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25930]: [2026-08-29 15:59:30 +0000] [25930] [INFO] Using work... sync
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25932]: [2026-08-29 15:59:30 +0000] [25932] [INFO] Booting wo... 25932
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25957]: [2026-08-29 15:59:30 +0000] [25957] [INFO] Booting wo... 25957
Aug 29 15:59:30 ip-10-0-10-206.ap-south-2.compute.internal gunicorn[25958]: [2026-08-29 15:59:30 +0000] [25958] [INFO] Booting wo... 25958

i-0fbb4135a4000ea0
PrivateIPs: 10.0.10.206
  
```

Figure 22 — ecommerce.service (Gunicorn/Flask) active and running

7.2 Background Worker Service

A second systemd service runs independently, continuously polling SQS for new orders to move into PROCESSING.



```

[ec2-user@ip-10-0-10-206 ~]$ sudo systemctl status ecommerce-worker.service --no-pager
● ecommerce-worker.service - CloudMart SQS Order Worker
   Loaded: loaded (/etc/systemd/system/eccommerce-worker.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-29 15:59:31 UTC; 38min ago
     Main PID: 25959 (python3)
       Tasks: 1 (limit: 4567)
      Memory: 35.0M
         CPU: 795ms
    CGroup: /system.slice/eccommerce-worker.service
            └─25959 /opt/eccommerce-app/venv/bin/python3 worker.py

Aug 29 15:59:31 ip-10-0-10-206.ap-south-2.compute.internal systemd[1]: Started ecommerce-worker.service - CloudMart SQS Order Worker.
[ec2-user@ip-10-0-10-206 ~]$
  
```

Figure 23 — ecommerce-worker.service (SQS consumer) active and running

7.3 Health Check & Logs

A direct HTTP check against the instance returns 200 OK, and the service logs show a clean startup with no errors.

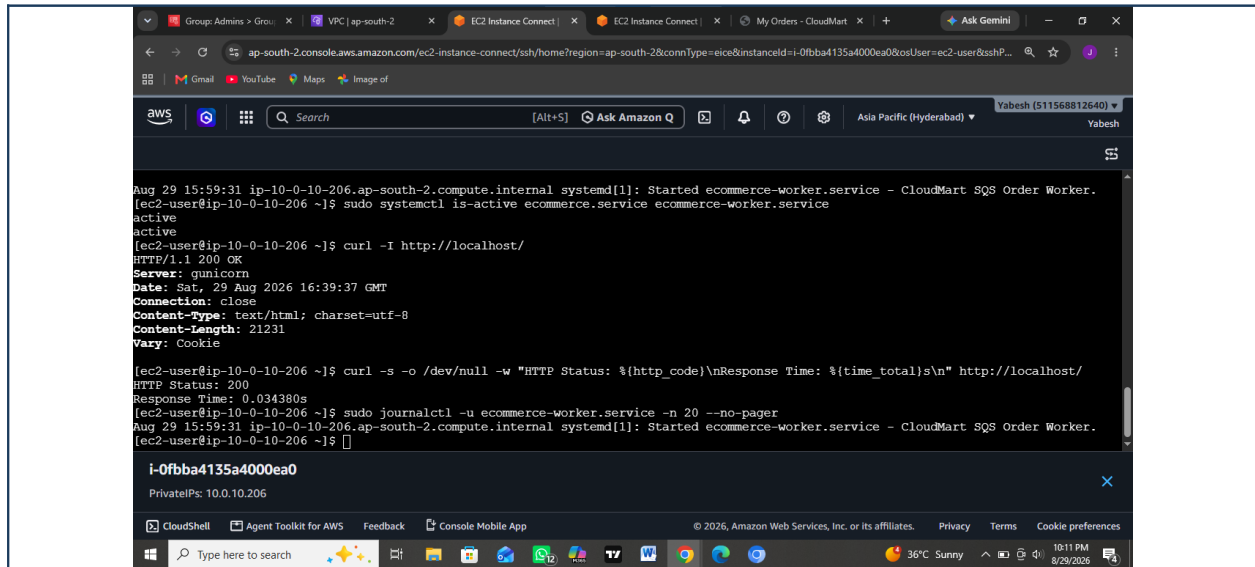


Figure 24 — HTTP health check returning 200 OK with clean service logs

8. Security Practices Applied

- Application tier and database run in private subnets with no direct internet exposure; only the ALB is internet-facing.
- Outbound internet access for private instances (package installs, AWS API calls) is routed through a NAT Gateway rather than a public IP.
- Authentication and password handling are delegated entirely to Amazon Cognito rather than custom code.
- Role-based access control is enforced server-side via a Cognito group ("Admins"), not a client-side flag.
- Product images are served through short-lived S3 presigned URLs; the media bucket itself remains private (Block Public Access enabled).
- The EC2 instance role follows least privilege, scoped to the specific S3 bucket and SQS queue the application needs.
- Sensitive configuration (database credentials, secret key) is stored outside source control and loaded at boot via an environment file.

9. Conclusion

CloudMart demonstrates a complete, highly available e-commerce workload on AWS: a fault-tolerant compute and data layer, decoupled order processing via SQS, secure managed authentication via Cognito, and a role-appropriate user experience for both customers and administrators. Infrastructure health, application behaviour, and the end-to-end order lifecycle — from checkout through admin approval to the customer seeing an updated status — have all been verified directly against the running system.